

Informe Actividad 3, Control 2

Institución: Departamento de Ciencias de la Computación, Facultad de Ciencias Físicas y Matemáticas, Universidad de Chile.

Curso: Redes (CC4303).

Sección: N°1.

Profesora: Ivana Bachmann

Fecha de entrega: 15/09/2026.

Estudiante: Diego Salinas.

Repositorio de GitHub: <https://github.com/diego051101/redes-diego-salinas-2026>

Declaración de uso de IA Se utilizó el agente Claude como asistente durante el desarrollo de esta entrega. Su función principal fue como guía en casos de bloqueo o no entendimiento de conceptos, así como ayuda en comandos específicos dentro de la terminal de VSCode. Se empleó, además, para crear algunos gráficos de acompañamiento visual para el presente informe (Figura 1 y Figura 2).

Cómo ejecutar (WSL / Linux)

No se usan librerías externas, solo socket, random, sys y time.

- **SocketTCP.py:** clase SocketTCP (TCP simplificado sobre sockets UDP con Stop & Wait).
- **cliente.py y servidor.py:** programas de la entrega de la tarea, donde el cliente envía un archivo y el servidor lo escribe en salida estándar.
- **pruebas_cliente.py y pruebas_servidor.py:** tests de send/recv del enunciado, más dos pruebas extra y el cierre.
- **archivo.txt:** archivo de prueba de 542 bytes.

Primero se levanta el servidor y luego, en otra terminal, el cliente. El servidor escribe en stdout solo el contenido recibido (los mensajes informativos y de debug van a stderr), así que se puede redirigir a un archivo y compararlo. El servidor queda esperando nuevos clientes hasta detenerlo con Ctrl+C:

```
cd control_2/actividad_3
python3 servidor.py localhost 8000 [-debug] [-perdida P] > recibido.txt
```

```
python3 cliente.py localhost 8000 [-debug] [-perdida P] < archivo.txt  
cmp archivo.txt recibido.txt && echo "archivo identico"
```

-debug activa el modo debug (muestra timeouts, retransmisiones, duplicados y casos borde a medida que se manejan). -perdida P simula pérdidas a mano: ese proceso descarta al azar el P% de los segmentos que envía; usándolo en ambos programas hay pérdidas en las dos direcciones. Para usar netem en lugar de pérdidas a mano:

```
sudo tc qdisc add dev lo root netem loss 20.0% delay 0.5s  
sudo tc qdisc del dev lo root netem
```

Tests del enunciado (también aceptan -debug y -perdida P):

```
python3 pruebas_servidor.py localhost 8001  
python3 pruebas_cliente.py localhost 8001
```

Flujo de funcionamiento

A continuación se describe paso a paso el flujo de funcionamiento del programa:

1. cliente.py lee el archivo completo desde entrada estándar como bytes, crea un SocketTCP() y llama a connect((IP, PUERTO)), que hace el lado cliente del 3-way handshake.
2. servidor.py hace bind((IP, PUERTO)) y espera en accept(). Al terminar el handshake, accept() retorna un nuevo SocketTCP asociado a otro puerto, que es el que se usa para recibir.
3. El cliente llama a send(contenido): el primer segmento lleva el largo del mensaje y después se envía el contenido en trozos de a lo más 16 bytes, cada uno con Stop & Wait.
4. El servidor llama a recv(64) repetidamente y escribe lo recibido. Igual que con los sockets de Python, recv retorna b"" cuando llega el FIN de la contraparte (y en ese caso ya cerró la conexión).
5. El cliente llama a close() (cierre del lado Host A) y el servidor completa el cierre (lado Host B) dentro de recv/recv_close(). Luego el servidor vuelve a accept() para atender otro cliente.

Estructura de los segmentos

Los headers se codifican directamente en bytes, con un tamaño fijo de 5 bytes como se indica en la siguiente tabla:

Bytes	Campo	Contenido
0	Flags	bit 0 = SYN, bit 1 = ACK, bit 2 = FIN

1 a 4	seq	número de secuencia, entero sin signo de 4 bytes (big endian)
5 o más	Datos	a lo más 16 bytes

parse_segment(bytes) retorna un diccionario con las llaves SYN, ACK, FIN, SEQ y DATOS, y create_segment(diccionario) hace lo inverso. El socket UDP interno usa siempre un buffer de $5 + 16 = 21$ bytes, independiente del buff_size de recv.

Números de secuencia y 3-way handshake

Se usa un único número de secuencia que ambos extremos mantienen sincronizado, como en la versión simplificada del video: el SYN lleva seq = x (x aleatorio entre 0 y 100), el SYN+ACK seq = x+1 y el ACK seq = x+2. Después, cada segmento de datos se envía con el seq actual y su ACK lleva seq + len(datos), que pasa a ser el nuevo seq en ambos lados. La Figura 1 muestra qué hacen connect y accept en cada paso.

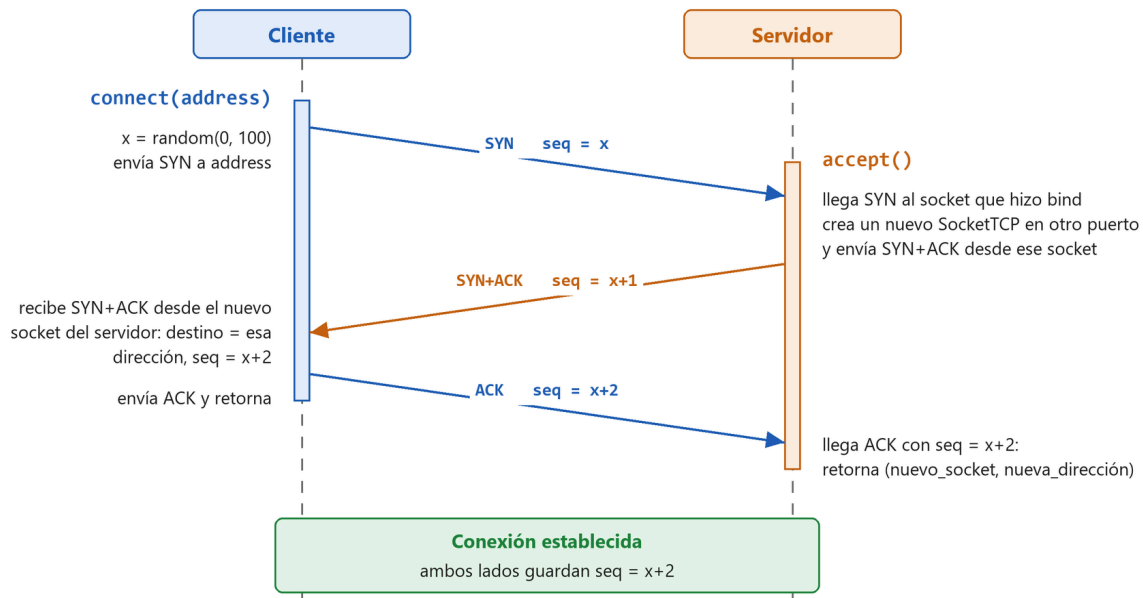


Figura 1: 3-way handshake y los pasos que realizan connect() y accept().

El nuevo socket se crea con `bind((IP, 0))`, así el sistema operativo le asigna un puerto libre distinto al del socket que escucha. El SYN+ACK se envía desde ese nuevo socket, por lo que el cliente conoce la nueva dirección gracias al `recvfrom` del SYN+ACK. Con pérdidas, connect reenvía el SYN y accept reenvía el SYN+ACK cada vez que se cumple el timeout.

send y recv con Stop & Wait

- **send(message):** envía primero un segmento con len(message) (8 bytes) y luego trozos de 16 bytes. Por cada segmento espera un ACK con seq = seq + len(datos); si se cumple el timeout (fijo, con settimeout) reenvía el mismo segmento. Los ACK con otro seq (atrasados o duplicados) se ignoran.
- **recv(buff_size):** si llega un segmento con el seq esperado guarda los datos, avanza el seq y responde ACK. Si llega un seq menor es una retransmisión (se perdió nuestro ACK): se vuelve a enviar el ACK sin guardar los datos otra vez.

Cierre de conexión

- **close() (Host A):** envía FIN (seq = s) y espera FIN+ACK (seq = s+1), reenviando el FIN en cada timeout. Tras 3 timeouts asume que la contraparte se cerró. Si llega el FIN+ACK, envía el último ACK (seq = s+2) tres veces, esperando un timeout entre cada envío. Al final cierra el socket UDP.
- **recv_close() (Host B):** espera el FIN (mientras tanto sigue respondiendo ACK a datos repetidos), responde FIN+ACK y espera el último ACK hasta 3 timeouts, reenviando FIN+ACK en cada timeout o si llega un FIN repetido. Tras el tercer timeout asume que la contraparte se cerró. Si el FIN llega mientras se está en recv, se ejecuta esta misma lógica y recv retorna b"".

Decisiones de diseño

- **Header binario de 5 bytes** (1 byte de flags + 4 bytes de seq) en lugar de texto: tiene tamaño fijo, así que es trivial separarlo de los datos. El largo del mensaje viaja como entero de 8 bytes en el área de datos del primer segmento, por lo que ese segmento también avanza el seq (en 8).
- **recv y buff_size (paso 5):** el socket guarda bytes_por_recibir (bytes del mensaje actual que aún no llegan) y datos_sin_entregar (bytes que llegaron pero todavía no se retornan). Si ambos están en cero, recv espera primero el segmento con el largo; si no, continúa el mensaje sin esperar un largo. Luego recibe segmentos hasta juntar buff_size bytes o completar el mensaje, es decir, retorna apenas len(recibido) >= min(largo restante, buff_size). Si un trozo de 16 bytes hace que se supere buff_size (por ejemplo con buff_size = 14), retorna solo buff_size bytes y el sobrante queda en datos_sin_entregar para la siguiente llamada, que lo entrega primero y solo espera segmentos si faltan bytes. Así nunca se retornan más de buff_size bytes y no se pierden datos.

- **Timeout fijo de 1.5 s** (TIMEOUT_SEGUNDOS en SocketTCP.py): es mayor que el RTT de 1 s que produce netem con delay 0.5s, para no retransmitir de más.
- **recv retorna b"" al recibir un FIN**, igual que los sockets TCP de Python. Así el servidor puede recibir un archivo de largo desconocido llamando a recv hasta obtener b"". recv_close() no hace nada si la conexión ya se cerró.
- **Pérdidas a mano:** además de netem, la variable global PROBABILIDAD_PERDIDA (opción -perdida P) descarta el P% de los segmentos antes de enviarlos. Como cliente y servidor usan la misma clase, se pierden segmentos en ambas direcciones (datos, ACK, SYN, FIN, etc.).
- **Modo debug** (-debug, variable global MODO_DEBUG): los mensajes van a stderr para no mezclarse con el archivo recibido.

Caso borde donde se pierde el último ACK del handshake

Si se pierde el ACK final, connect() ya retornó y el cliente empieza con send(), pero el servidor sigue en accept() esperando ese ACK. Si no se maneja, accept() ignora los datos y reenvía el SYN+ACK, mientras send() ignora el SYN+ACK y reenvía los datos: ninguno de los dos avanza. La Figura 2 muestra el caso junto a la solución implementada.

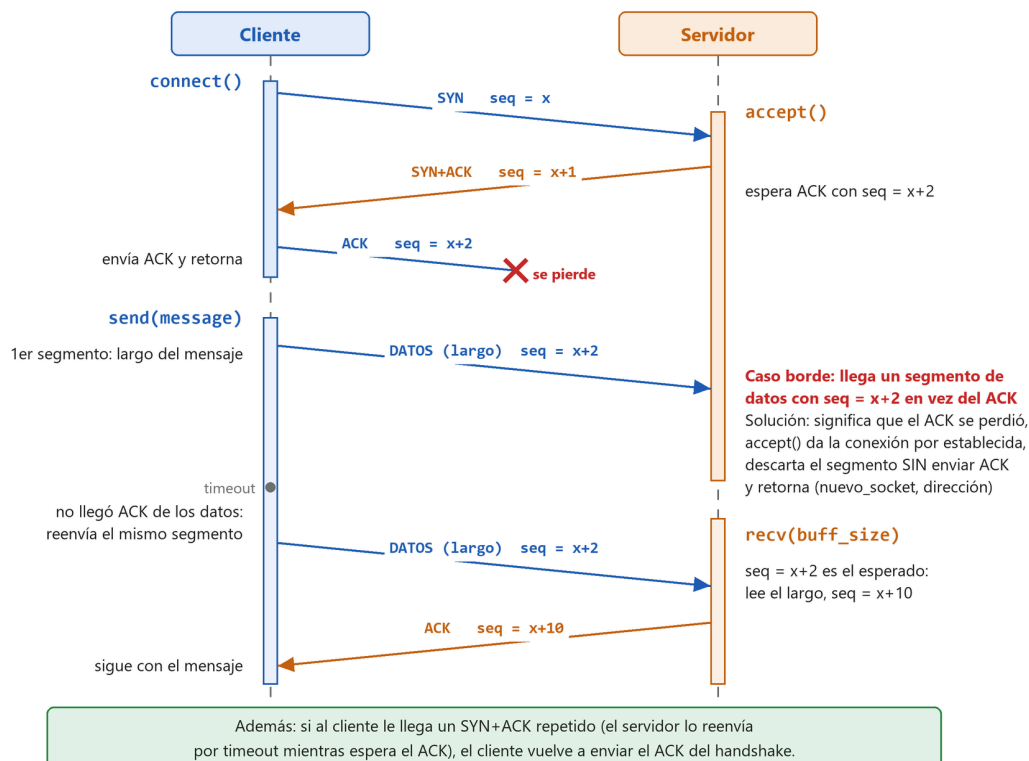


Figura 2: pérdida del último ACK del handshake y solución implementada.

- En `accept()`, si llega desde el cliente un segmento sin SYN ni ACK con `seq = x+2` (datos o FIN), significa que el cliente ya terminó el handshake, es decir, el ACK se perdió. `accept()` da la conexión por establecida y retorna, **descartando ese segmento sin responder ACK**: el cliente lo reenvía por timeout y lo recibe `recv` (o `recv_close`) de forma normal. Cuesta un timeout, pero evita tener que guardar segmentos entre funciones.
- Además, si al cliente le llega un SYN+ACK repetido (en `send`, `recv` o `close`), vuelve a enviar el ACK del handshake. Esto cubre el caso en que el cliente no envía nada después de conectarse (por ejemplo, si primero espera recibir).

Otros casos borde encontrados

- **SYN repetidos**: si el SYN+ACK se pierde o se atrasa, el cliente reenvía el SYN al socket que escucha. Esas copias quedarían encoladas y el siguiente `accept()` crearía una conexión fantasma. Para evitarlo, el socket que escucha guarda los pares (dirección, `seq` del SYN) ya aceptados e ignora sus repeticiones.
- **Se pierde el ACK del último trozo de datos**: `recv` ya retornó, pero el emisor reenvía el trozo. La copia la atiende la siguiente llamada a `recv` o `recv_close` (`seq` menor al esperado, así que se reenvía el ACK). Por eso el receptor debe seguir llamando a `recv` o `recv_close` después de recibir un mensaje.
- **Se pierden los 3 FIN de `close()`**: el Host A se cierra tras 3 timeouts como pide el enunciado, pero el Host B nunca recibió un FIN y queda esperando en `recv/recv_close` indefinidamente (no puede distinguir a una contraparte lenta de una que ya se cerró). El enunciado solo garantiza el cierre de B una vez que recibe un FIN, así que se dejó así. Pasó en 1 de las pruebas con 30% de pérdida (probabilidad $0.3^3 \approx 2.7\%$).
- **Llegan datos nuevos durante `recv_close()`**: no son un duplicado, pero la aplicación ya no está leyendo. Se responde ACK para no bloquear al emisor y los datos se descartan (se avisa en modo debug).
- **Windows**: `recvfrom` puede lanzar `ConnectionResetError` si antes se envió algo a un puerto ya cerrado; se ignora y se sigue esperando.
- **Limitaciones**: el `seq` es de 4 bytes (hasta ~4 GB enviados por conexión). `connect()` reintenta indefinidamente si no hay servidor, y `accept()` también si el cliente desaparece a mitad del handshake (el enunciado no pide un máximo de reintentos ahí). El servidor atiende a un cliente a la vez.

Pruebas realizadas y resultados

Todas las pruebas se hicieron en WSL (Ubuntu, Python 3.12) sobre localhost. "Transferencia" es cliente.py < archivo.txt hacia servidor.py (542 bytes, comparados con cmp); "tests" son pruebas_cliente.py/pruebas_servidor.py.

A continuación se detallan las pruebas en una tabla:

Escenario	Prueba	Resultado
Sin pérdidas	Tests 1 a 5 + cierre	5/5 Passed, ambos lados cierran
Sin pérdidas	Transferencia (1 corrida)	Archivo idéntico, 5 s (4.5 s son los 3 ACK finales de close)
Pérdida a mano 20% (ambos lados)	Transferencia (5 corridas)	5/5 archivos idénticos, 38 a 54 s, 13 a 23 retransmisiones por corrida
Pérdida a mano 35% (ambos lados)	Transferencia (6 corridas)	6/6 archivos idénticos, 72 a 110 s
Pérdida a mano 40% (ambos lados)	Transferencia (3 corridas)	3/3 archivos idénticos, 87 a 142 s, 45 a 78 retransmisiones por corrida
Pérdida a mano 30% (ambos lados)	Tests 1 a 5 (3 corridas)	5/5 Passed en las 3 corridas; el cliente siempre cierra
netem loss 20% delay 0.5s	Transferencia (2 corridas)	2/2 archivos idénticos, 68 y 81 s, ambos lados cierran
netem loss 20% delay 0.5s	Tests 1 a 5 (1 corrida)	5/5 Passed en 96 s, ambos lados cierran

- **Test 3 y test 4 (paso 5):** con buff_size = 14 el mensaje de 19 bytes se recibe como 14 + 5 bytes. Con un mensaje de 2n = 42 bytes y dos llamadas a recv(21) (21 no es múltiplo de 16) cada llamada retorna exactamente 21 bytes y la concatenación es igual al original.
- **Casos borde observados en el modo debug:** el ACK final del handshake se perdió en 6 de las 14 transferencias con pérdida a mano y en las 2 con netem, y en todas accept() lo detectó al llegar el primer segmento de datos y la transferencia terminó bien. También se vio que el socket que escucha descartó SYN repetidos (una vez con pérdida a mano y una con netem, cuando se perdió un SYN+ACK y el cliente reenvió el SYN) y un recv_close que cerró tras 3 timeouts porque se perdieron los últimos ACK.

Extracto real del modo debug (stderr) de una transferencia con -debug -perdida 35 en ambos programas, en la que se perdió el ACK del handshake: el cliente ya se cree

conectado y envía el largo con seq = 90; accept() lo detecta, descarta el segmento y retorna; el cliente lo reenvía por timeout y recv vuelve a responder ACK a las copias repetidas, porque sus ACK también se perdían.

```
# cliente.py
```

```
(debug) connect: se envia SYN con seq = 88
```

```
(debug) connect: SYN+ACK recibido desde ('127.0.0.1', 37007) -> se envia ACK  
con seq = 90
```

```
(debug) (perdida simulada) no se envia [ACK seq=90]
```

```
(debug) send: enviando mensaje de 542 bytes
```

```
(debug) send: timeout esperando ACK 98 -> reenviando segmento seq = 90
```

```
# servidor.py
```

```
(debug) accept: SYN recibido desde ('127.0.0.1', 55461) con seq = 88
```

```
(debug) accept: se envia SYN+ACK con seq = 89 desde ('127.0.0.1', 37007)
```

```
(debug) accept: llego [DATOS seq=90 datos=b'\x00\x00\x00\x00\x00\x00\x02\x1e']  
en vez del ACK -> se perdio el ACK del handshake, se da la conexion por  
establecida y se descarta el segmento (el cliente lo reenviara)
```

```
(debug) (perdida simulada) no se envia [ACK seq=98]
```

```
(debug) recv: se va a recibir un mensaje de 542 bytes
```

```
(debug) recv: segmento duplicado seq = 90 -> se reenvia ACK seq = 98
```